

03. フロントエンド設計書

Version 1.0 (FIX) / 2026-08-17 / React (TypeScript) + Vite

1. 画面一覧

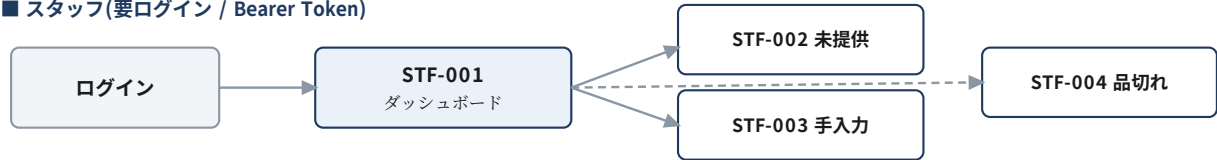
ID	画面名	URL	認証	概要
CST-001	顧客メニュー	/?table={table_id}	session	メニュー閲覧・カート追加
CST-002	カート	(CST-001上のModal)	session	内容確認・注文送信
CST-003	注文履歴	/history?order_id={id}	session	提供状況確認・お会計呼出
STF-000	ログイン	/staff/login	—	スタッフ認証
STF-001	卓ダッシュボード	/staff	Bearer	卓状態一覧・開栓・会計クリア
STF-002	未提供リスト	(STF-001上 右ペイン)	Bearer	提供済/Undo/取消
STF-003	手入力	(STF-001上のModal)	Bearer	カスタム商品の追加
STF-004	品切れ管理	/staff/products	Bearer	品切れの設定・解除

2. 画面遷移

■ 顧客(認証なし / session_tokenで検証)



■ スタッフ(要ログイン / Bearer Token)



※ スタッフ画面は全て /staff 配下。ダッシュボードを起点に各機能をModal/サブ画面として開く。

図1 画面遷移図

3. ワイヤフレーム

3.1 顧客画面

CST-001 メニュー

T1 (5名) ご注文

シーシャ

ドリンク

フード

商品名
¥1,650 (税込)

追加

商品名
¥1,650 (税込)

追加

商品名
¥1,650 (税込)

追加

商品名
¥1,650 (税込)

追加

カートを見る (2点)

CST-002 カート

カート内容

商品名 × 1
¥1,650

— +

商品名 × 1
¥1,650

— +

商品名 × 1
¥1,650

— +

参考小計(税込)

¥4,950

お会計はレジにて確定します

この内容で注文する

CST-003 注文履歴

ご注文の状況

商品名 × 1
¥1,650

提供済

商品名 × 1
¥1,650

提供済

商品名 × 1
¥1,650

準備中

商品名 × 1
¥1,650

準備中

参考小計(税込)

¥6,600

追加注文

お会計呼出

図2 顧客画面(スマートフォン想定)

顧客画面の設計方針

アプリのインストールも会員登録も不要で、QRコードを読み取ればすぐ注文できることを最優先とする。

金額はすべて税込で表示し、カート・履歴には「最終的なお会計はレジにて確定します」の注記を常に表示して、実際の会計金額との差異による誤解を防ぐ([00]2.2節)。

3.2 スタッフ画面

STF-001 卓ダッシュボード

手入力 | 品切れ管理 | ログアウト

フロアマップ (座標はDBのpos_x / pos_yから描画)

T1 5名
使用中
¥6,600 / 未提供2
会計クリア

T2 4名
呼出中
¥4,950 / 未提供0
会計クリア

T3 2名
空席
—
開栓

T4 2名
空席
—
開栓

C1 1名
使用中
¥1,650 / 未提供1
会計クリア

C2 1名
空席
—
開栓

STF-002 未提供リスト

T1 シーシャ × 1
19:42 受付
提供済

T1 モヒート × 1
19:42 受付
提供済

C1 ナッツ盛り × 1
19:42 受付
提供済

T2 コーラ × 1
19:42 受付
Undo

※ 提供済は一定時間表示し、Undoで未提供へ戻せる

図3 スタッフ画面(タブレット横向き想定)

スタッフ画面の設計方針

1画面で「どの卓が呼んでいるか」「何が未提供か」が同時に見えることを最優先とする。フロアマップと未提供リストを左右に並べ、画面遷移なしで日常業務が完結するようにしている。

タップ対象は指で確実に押せる大きさを確保し、提供済の操作は必ずUndoできるようにすることで、誤タップによる業務停止を防ぐ。

4. 画面別詳細仕様

CST-001 顧客メニュー /?table={table_id}

- ・ 起動時に GET /api/v1/tables/{table_id}/session を実行し、order_id と session_token を取得する。session_token は sessionStorage に保持し(6章)、以降の全リクエストで X-Session-Token ヘッダーに付与する。
- ・ 未開栓(403 TABLE_NOT_ACTIVE)の場合は「ただいまご利用いただけません。スタッフにお声がけください」を表示する。
- ・ GET /api/v1/products で品切れを除いたメニューを取得し、category ごとにタブ表示、sort_order 昇順で並べる。
- ・ 価格表示: products.price は税込価格のため、換算せずそのまま表示する。
- ・ カートへの追加は Zustand のローカル状態にのみ反映し、この時点ではAPIを呼ばない。

CST-002 カート(Modal)

- ・ カート内商品の数量増減・削除を行い、税込合計(参考小計)を表示する。カテゴリのタブは横スクロール、タブ内は subcategory ごとに見出しを挟んで区切る(アルコールが50品あり、平坦なリストでは探せないため)。
- ・ 「この内容で注文する」で POST /api/v1/orders/{order_id}/items へ一括送信する(X-Session-Token必須)。
- ・ 送信成功後はカートのを空にし、CST-003(注文履歴)へ遷移する。
- ・ 二重送信防止のため、送信中はボタンを非活性にする。

CST-003 注文履歴 /history?order_id={id}

- ・ URLパラメータに order_id を含め、リロード時の状態復元を可能にする。ただし session_token はURLに含めず sessionStorage から復元する(6章)。
- ・ SWRにより8秒間隔でポーリングし、提供状況(準備中 / 提供済)を更新する。
- ・ cancelled の明細は表示せず、参考小計の集計対象からも除外する。
- ・ 「お会計呼出」で POST /api/v1/orders/{order_id}/call を実行する。実行後はボタンを「スタッフをお呼びしました」の表示に変え、連打を防ぐ。

STF-001 卓ダッシュボード /staff

- ・ GET /api/v1/tables のレスポンス(pos_x, pos_y)に基づき、CSS absolute等で卓を配置する。座標のハードコードは行わない。
- ・ 基準キャンバス 450 × 490 を画面幅に応じて比例縮小して描画する。キッチン・カウンター・冷蔵庫・トイレ等の什器は静的な背景として描画する(DB管理外)。
- ・ SWRにより4秒間隔でポーリングし、卓の状態を更新する。
- ・ 卓の状態は3種類: 空席(白) / 使用中(青) / 呼出中(赤・ドット表示)。
- ・ 空席の卓は「開栓」、使用中・呼出中の卓は「会計クリア」ボタンを表示する。
- ・ 開栓時に409 (TABLE_ALREADY_ACTIVE) を受け取った場合は「既に開栓済みです」を表示し、一覧を再取得する。
- ・ 会計クリアは誤操作の影響が大きいいため、確認ダイアログを挟む。

STF-002 未提供リスト(STF-001上 右ペイン)

- ・ GET /api/v1/staff/pending_items で全卓の未提供明細を受付時刻(created_at)昇順で取得する。
- ・ 「提供済」ボタンのタップで PATCH /api/v1/order_items/{item_id}/status (status: served) を実行する。
- ・ 誤タップ対策として、提供済にした明細も一定時間リストに残し、「Undo」ボタンで status: pending に戻せるようにする。
- ・ 客都合等で明細を取消す場合は、長押し等の別操作で status: cancelled を実行する(誤操作防止のため「提供済」の単純タップとは明確に区別する)。
- ・ SWRにより4秒間隔でポーリングする。

STF-003 手入力(Modal)

- 対象の卓を選択し、custom_name / price(税込)を入力してPOST /api/v1/orders/{order_id}/custom_items へ送信する。
- 価格は税込で入力する。GUESTジンのような変動価格の商品も、この画面から実際の金額で追加できる。

STF-004 品切れ管理 /staff/products

- GET /api/v1/staff/products で全商品(品切れ含む)を一覧表示する。
- 各商品のトグルスイッチから PATCH /api/v1/staff/products/{id}/sold_out を実行し、品切れ状態を即時反映する。
- 楽観的更新(Optimistic UI)を行い、失敗時は元の状態に戻してエラーを表示する。

5. 状態管理の役割分担

ライブラリ	役割	扱うデータの例
SWR	Backend APIから取得するサーバー状態のキャッシュ・再検証・ポーリング	products, tables, 注文履歴, 未提供リスト
Zustand	サーバーに保存されるまでの一時的なUI状態	カートの中身, Modalの開閉
sessionStorage	セッション情報の永続化(タブを閉じるまで)	order_id, session_token
localStorage	スタッフのログイン状態の保持	JWT

原則として、サーバーから取得できるデータをZustandに複製しない。状態の二重管理は不整合の温床となるため、サーバー状態はSWRのキャッシュを唯一の情報源とする。

6. セッション情報の保持方針

- session_token は sessionStorage に order_id と紐づけて保持し、顧客系リクエストで X-Session-Token ヘッダーに付与する。
- URLクエリパラメータには含めない。URLはブラウザ履歴・共有・アクセスログに残るため、秘密情報を書けない方針とする。
- sessionStorageはXSSの影響を受け得るため、React標準のエスケープを前提とし、dangerouslySetInnerHTML を使用しないことを開発規約とする([05]参照)。
- ブラウザ/タブを閉じるとsessionStorageは失われる。再訪時は再度QRコードを読み取り、session取得APIから取り直す運用とする。
- 同一卓の複数人が別々の端末からQRコードを読み取った場合、いずれも同じ order_id / session_token を取得するため、全員が同じ伝票に注文できる(仕様として許容する)。

7. エラーハンドリング方針

[01]8章の共通エラーフォーマット(error.code)に基づき、APIクライアント層で一元的に処理する。

code	FEでの挙動
TABLE_NOT_ACTIVE	「ただいまご利用いただけません」の案内を表示
SESSION_EXPIRED	sessionStorageを破棄し、QRコードの再読み込みを促す案内を表示
UNAUTHORIZED	JWTを破棄し、スタッフをログイン画面へリダイレクト
TABLE_ALREADY_ACTIVE	「既に開栓済みです」を表示し、一覧を再取得
INVALID_STATUS_TRANSITION	「他の端末で操作された可能性があります」を表示し、一覧を再取得
RATE_LIMITED	しばらく時間をおいて再試行するよう案内
VALIDATION_ERROR / INTERNAL_ERROR	汎用エラー表示(トースト)

8. ディレクトリ構成

機能単位(features)でディレクトリを分割し、顧客用とスタッフ用のコードを明確に分離する。共通処理のみ shared に置く。

```
frontend/
├── src/
│   ├── main.tsx      エントリポイント
│   ├── App.tsx       ルーティング定義
│   ├── features/
│   │   ├── customer/  顧客向け機能
│   │   │   ├── MenuPage.tsx    CST-001
│   │   │   ├── CartModal.tsx   CST-002
│   │   │   ├── HistoryPage.tsx CST-003
│   │   │   └── useCartStore.ts  Zustand(カート状態)
│   │   └── staff/          スタッフ向け機能
│   │       ├── LoginPage.tsx   STF-000
│   │       ├── DashboardPage.tsx STF-001
│   │       ├── FloorMap.tsx    卓の座標配置描画
│   │       ├── PendingList.tsx STF-002
│   │       ├── CustomItemModal.tsx STF-003
│   │       └── ProductsPage.tsx STF-004
│   ├── shared/
│   │   ├── api/
│   │   │   ├── client.ts      fetchラップ(ヘッダ付与・エラー整形)
│   │   │   └── types.ts       APIの型定義
│   │   ├── session.ts         sessionStorage/localStorage操作
│   │   ├── price.ts           税込計算(単一実装)
│   │   └── components/        Button, Modal, Toast 等
│   └── styles/
├── .env.local                 VITE_API_BASE_URL
├── index.html
├── package.json
└── vite.config.ts
```

価格計算を1箇所に集約する

金額の整形(3桁区切り・通貨記号)は shared/price.ts の関数のみに実装し、各画面から必ずこれ呼び出す。価格自体は税込で保持しているため換算は不要だが、書式が画面ごとにばらつくと同じ金額が違って見えるため、1箇所に集約する。

9. 変更履歴

版	主な変更内容
V3(ドラフト)	初版。Supabase Realtime直接購読を前提。
V4(レビュー反映)	X-Session-Token付与を明記。Realtime購読をSWRポーリングへ変更。品切れ管理画面(STF-004)、明細取消の導線、エラーハンドリング方針を追加。
V1.0 (FIX)	画面遷移図・ワイヤーフレーム(顧客/スタッフ)を追加。画面一覧を表形式に整理。ディレクトリ構成、状態管理の役割分担、複数端末利用時の挙動を明記。